

Anatomie d'une commande Mongosh CLI

Hello friend,

Pour exécuter une requête directement depuis le terminal Linux sans ouvrir l'interpréteur interactif, la structure s'articule toujours de la même manière :

```
mongosh --eval "db.getSiblingDB('nom_base').nom_collection.methode({filtre}, {options})"
```

- `mongosh` : Lance l'outil de ligne de commande MongoDB.
- `--eval "..."` : Permet de passer une instruction unique à exécuter directement depuis le terminal, puis de quitter.
- `db.getSiblingDB('nom_base')` : Permet de cibler explicitement la base de données sans avoir à faire de `use` au préalable.
- `.nom_collection` : Sélectionne la collection ciblée à l'intérieur de cette base.
- `.methode(...)` : L'action ou l'opérateur MongoDB à appliquer (`countDocuments()`, `find()`, `findOne()`, `updateOne()`, etc.).

Construire ses propres requêtes pas à pas

- **Étape 1 (Le ciblage)** : Commence toujours par `mongosh --eval "db.getSiblingDB('nom_base').nom_collection"` pour pointer exactement vers la bonne table.
- **Étape 2 (L'action de lecture)** : Ajoute `.find()` pour lister, `.findOne()` pour un seul élément, ou `.countDocuments()` pour compter.
- **Étape 3 (Les critères optionnels)** : Glisser les filtres en JSON à l'intérieur des parenthèses, comme `{ annee: 2020 }` ou `{ auteur: "Nom" }`.
- **Étape 4 (Les modificateurs de tri)** : Enchaîner si besoin avec `.sort({ annee: 1 })` pour l'ordre croissant ou `.limit(1)` pour restreindre l'affichage.

Boîte à outils des filtres et opérateurs MongoDB

- **Filtre par égalité (valeur exacte)**
 - *Syntaxe* : `{ "champ": "valeur" }` ou `{ "annee": valeur }`
- **Opérateurs de comparaison (`$gt`, `$lt`)**

- *Syntaxe* : `{ "champ": { "$gt": valeur } }` (strictement supérieur) ou `{ "champ": { "$lt": valeur } }` (strictement inférieur)
- **Recherche dans un tableau ou champ complexe**
 - *Syntaxe* : `{ "nom_tableau": "valeur_recherchée" }` (pour cibler un élément spécifique au sein d'une liste)
- **Pagination et décalage (`skip` et `limit`)**
 - *Syntaxe* : `.skip(nombre_a_sauter).limit(nombre_a_afficher)` (à enchaîner après un `.find()`)
- **Tri des résultats (`sort`)**
 - *Syntaxe* : `.sort({ "champ": 1 })` (ordre croissant) ou `.sort({ "champ": -1 })` (ordre décroissant)
- **Mise à jour et modification de documents (`$set`)**
 - *Syntaxe* : `updateOne({ filtre }, { "$set": { "nouveau_champ": valeur } })` ou `updateMany(...)`